SAE 1.2 – Comparaison d’approches algorithmiques

Table des matières

Introduction....1
Lancement du projet....2
Planification et organisation du travail en équipe....2
Préparation technique de l’outil de travail....2
Déroulement du projet : exécution de tests unitaires des fonctions et procédures puis rapports de test....3
REWIND....3
ADD....4
REMOVE....6
BOOLEAN « HASNEXT »....8
SIZE....10
GET....11
SET....12
NEXT....13
BOOLEAN « HASPREVIOUS » et « PREVIOUS »....13
ADDITER....14
Conclusions....15
Table des figures....15

Introduction

Nous sommes Benjamin BOUTOUT, Louis CARON et Marie TIERS, groupe d’apprentis développeurs impliqués dans un nouveau projet de SAE 1.2 du BUT 1 informatique.

La mission qui nous incombe aujourd’hui est de nous familiariser avec l’utilisation des fonctions et des procédures dans le langage Java. Nous allons la mener à bien grâce à l’étude approfondie de ce qui est cité dans l’énoncé donné. Le voici :

> *« Le fonctionnement et les performances d’une structure de données que vous ne connaissez pas et qui est cachée derrière un ensemble de procédures et de fonctions ».*

Lancement du projet

Planification et organisation du travail en équipe

Un squelette de programme Java nous a été fourni dans l'énoncé, afin de nous aider à nous mettre en route plus rapidement. Nous le collons tel quel dans le fichier Java développé conjointement par les membres de notre groupe.

Pour ce travail d'équipe, il nous a semblé indispensable de travailler à trois de front. Nous nous sommes donc partagés les tâches afin de gagner en efficacité (rapidité et productivité en code fonctionnel). Dans notre cas présent, une tâche équivaut à l'étude d'une unique procédure ou d'une unique fonction. L'ensemble de ces tâches individuelles sont réparties entre deux testeurs, et le troisième et dernier membre rédige ce présent compte-rendu sous la supervision des testeurs.

Préparation technique de l'outil de travail

Nous utilisons l'IDE Visual Studio Code.

Nous importons dans notre programme en Java une librairie (fichier Data.jar) qui contient une liste d'API préconçues avec des noms explicites sur leur rôle (Cf. énoncé pour en retrouver la liste). Ces dernières vont venir modifier d'une manière particulière les valeurs et le nombre de ces valeurs qui sont stockées dans une structure de données. Le type de structure de données choisi est le tableau à une dimension. Notre enjeu est de découvrir un maximum de manipulations possibles sur cette structure de données et sur les valeurs qu'elle contient. Le détail de ces manipulations sera expliqué dans la suite de ce compte-rendu.

Déroulement du projet : exécution de tests unitaires des fonctions et procédures puis rapports de test

REWIND

La fonction *rewind*, comme indiqué sur le document, sert à replacer l'itérateur au début de la structure, avant le premier élément. Donc nous avons choisi de créer une liste de cinq nombres et de faire un test de *rewind* à partir de différents emplacements de cette liste :

1. Au début,
2. Au milieu et
3. A la fin.

Ce test consiste à répéter trois fois la valeur choisie et ensuite d'utiliser *rewind*. Alors *rewind* entre dans la boucle.

Mais nous constatons que la durée d'exécution est identique, aussi bien quand *rewind* est utilisé au début qu'au milieu ou qu'à la fin. Il est compris entre 0.0ms et 1.0ms. On peut par exemple utiliser *rewind* au début d'un programme pour être sûr qu'aucune trace de notre ancien test n'existe.

Illustration du code ci-après (fig. 1 : code avec *rewind*) :

TestRewind.java > ...

```java
import java.io.*;
import java.util.*;
class TestRewind extends Data {
void run() {
double debut, fin;
add(5);
add(12);
add(6);
add(14);
add(7);
rewind();
debut = System.currentTimeMillis();
for(int i=0; i<3; i++) {
System.out.println(get(0));
rewind();
}
fin = System.currentTimeMillis();
System.out.println("Temps passé : " + (fin - debut) + " ms");
debut = System.currentTimeMillis();
for(int i=0; i<3; i++) {
System.out.println(get(2));
rewind();
}
fin = System.currentTimeMillis();
System.out.println("Temps passé : " + (fin - debut) + " ms");
for(int i=0; i<3; i++) {
System.out.println(get(4));
rewind();
}
fin = System.currentTimeMillis();
System.out.println("Temps passé : " + (fin - debut) + " ms");
}
Run | Debug
public static void main(String[] args) {
new TestRewind().run();
}
}
```

ADD

Une boucle est employée pour remplir un tableau[1] avec des valeurs aléatoires. Les valeurs s'ajoutent à la fin du tableau.

Quand le test de *add* est exécuté plusieurs fois, la durée d'exécution est constante et quasiment instantanée (très rapide). Nous remarquons qu'il dépend du nombre d'éléments contenus dans le tableau. Pour 10 éléments, la durée d'exécution est de 1ms.

1 Ici, on admet que « tableau » est synonyme de « liste ». Le testeur a trouvé davantage de facilité à visualiser la liste de valeurs sous forme de tableau, plus imagée.

Illustration du code ci-après (fig. 2 : code avec *add*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    // ajout de 3 éléments
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }
    rewind();

    debut = System.currentTimeMillis();
    for (int i = 0; i < size(); i++) {
        System.out.println(get(i));
    }

    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

REMOVE

Nous admettons qu'une liste de nombres aléatoires existe déjà. Le test consiste à supprimer les nombres impairs de cette liste à l'aide de *remove*. *Remove* permet d'enlever un élément avec, en paramètre, sa valeur dans le tableau. Toutes les autres positions qui ont été impactées par l'appel de *remove* se décalent au sein de la liste. Comme pour *add* précédemment, le temps d'exécution affiché correspond au temps nécessaire pour parcourir tous les éléments de la liste, donc il dépend du nombre d'éléments que celle-ci contient. Pour 10 éléments, la durée d'exécution est de moins d'une 1ms.

Attention, pour supprimer une valeur de notre structure de données, ce n'est pas l'indice de sa position qu'il faut mettre en paramètre de la procédure. Exemple : 3 est la valeur à l'index 2, quand on remove 3 ça supprimer la valeur 3 et non pas la valeur en index 3.

Cas de test non exécuter : quand il y a plusieurs occurrence d'une même valeur dans notre structure de données, est-ce que *remove* supprime la première valeur, la dernière valeur, une valeur à une position aléatoire, ou bien chacune des occurrences de cette valeur ? Nous déduisons que seule la première occurrence est supprimée, mais avec du temps de travail supplémentaire et cette tâche marquée « to do » nous pourrons effectuer ce test en priorité une prochaine fois.

Illustration du code ci-après (fig. 3 : code avec *remove*) :

```
    // ajout de 3 éléments
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }
    rewind();
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();
    System.out.println(x: "On enlève les nombres impaires");
    rewind();
    debut = System.currentTimeMillis();
    for (int i = 0; i < size(); i++) {
        if ((get(i) % 2) == 1) {
            remove(get(i));
            i--;
        }
    }
    fin = System.currentTimeMillis();
    remove(size() - 1);
    rewind();

    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();

    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

BOOLEAN « HASNEXT »

Le booléen hasNext est, comme le document nous l'indique, une variable qui nous retourne « true » s'il y a au moins un élément disponible après l'itérateur. Par exemple : sur une liste de deux éléments, le booléen nous retournera « true » pour le premier élément de la liste, mais répondra « false » pour le deuxième.

Prenons une liste de cinq éléments et plaçons un itérateur en position *n*. Tant qu'une valeur existe en position *n* dans notre structure de données et qu'elle précède une valeur existante en position *n+1*, hasNext renvoie « true » et l'itérateur passe à la position *n*+1, et ainsi de suite jusqu'à la fin de la liste. Par exemple : pour le premier élément de la liste, *n* sera 5 et il écrira les éléments de la liste jusqu'à ce qu'il n'y ait plus d'élément disponible juste après. Puisque la liste contient 5 éléments, il nous le fait 5 fois.

La durée moyenne d'exécution est de 8ms, même si nous sommes tombés une fois sur 10ms.

Voici notre liste de valeurs : 5 12 6 14 7. Voici ce qu'affichera le programme aux exécutions successives de *add* (fig. 4 : résultats de *hasNext*) :

```
5 5
5 12
5 6
5 14
5 7
```

Illustration du code ci-après (fig. 5 : code avec *hasNext*) :

TesthasNext.java > TesthasNext

```java
import java.io.*;
import java.util.*;
class TesthasNext extends Data {
void run() {
    boolean hasNext = true;
double debut, fin;
add(5);
add(12);
add(6);
add(14);
add(7);
rewind();
debut = System.currentTimeMillis();
while (hasNext(0)) {
    int n = next(0);
rewind(1);
while (hasNext(1)) {
System.out.println(n + " " + next(1));
}}
fin = System.currentTimeMillis();
System.out.println("Temps passé : " + (fin - debut) + " ms");}
Run | Debug
public static void main(String[] args) {
new TesthasNext().run();
}
}
```

SIZE

Prenons un tableau avec des valeurs aléatoires. *Size* nous permet d'obtenir le nombre d'éléments présents dans le tableau.

Durée d'exécution : 0ms.

Illustration du code ci-après (fig. 6 : code avec *size*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }
    rewind();

    debut = System.currentTimeMillis();
    System.out.println(size());

    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

GET

Prenons un tableau avec des valeurs aléatoires. Dans une boucle for, grâce à *get*, on affiche tous les éléments du tableau, les uns après les autre.

Durée d'exécution : 0ms.

Illustration du code ci-après (fig. 7 : code avec *get*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }
    rewind();
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();
    rewind();
    debut = System.currentTimeMillis();
    System.out.println(get(5)); // Get the element 6 of the tab
    System.out.println(get(0)); // get the fisrt element
    System.out.println(get(size() - 1)); // get the last elemen

    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

SET

Dans un tableau avec des valeurs aléatoires préremplies, on remplace la valeur à la position indiquée en premier paramètre de la procédure par la valeur qui est donnée en deuxième paramètre. Cela modifie l'élément à la position indiquée.

Durée d'exécution : 0ms.

Illustration du code ci-après (fig. 8 : code avec *set*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }

    rewind();
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();

    rewind();

    debut = System.currentTimeMillis();
    set(0, 20); // set 20 in the first element
    set(size() - 1, 50); // set 50 in the last element
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();
    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

NEXT

Comme nous indique le document, *next* nous renvoie l'élément disponible juste après l'itérateur puis avance à la position suivante. Nous l'avons utilisé dans le test de *hasNext* puisque les deux sont étroitement liés.

BOOLEAN « HASPREVIOUS » et « PREVIOUS »

Dans un tableau avec des valeurs aléatoires préremplies, on utilise une boucle for pour déplacer l'itérateur vers la droite (vers la fin de la liste). On utilise la fonction *hasPrevious* pour vérifier s'il existe des éléments à gauche (avant) l'itérateur et, au moyen de *previous*, on déplace l'itérateur vers la gauche si la condition est vraie.

Illustration du code ci-après (fig. 9 : code avec *hasPrevious* et *previous*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }

    rewind();
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();

    rewind();

    debut = System.currentTimeMillis();
    for (int i = 0; i < 7; i++) {
        next();
    }
    int nb = 0;
    while (hasPrevious()) {
        nb++;
        previous(); // reculer le cursor vers la droite
    }
    System.out.println(nb);
    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

ADDITER

Au moyen de la boucle for, on déplace l'itérateur à la position *n* et on utilise *addIter* pour placer la valeur en paramètre de cette fonction à l'endroit où l'itérateur est placé.

Illustration du code ci-après (fig. 10 : code avec *addIter*) :

```
void run() {
    double debut, fin;
    int aleatoire;
    for (int i = 0; i < NB; i++) {
        aleatoire = rand.nextInt(MAX);
        add(aleatoire);
    }

    rewind();
    for (int i = 0; i < size(); i++) {
        System.out.print(get(i) + " ");
    }
    System.out.println();

    rewind();

    debut = System.currentTimeMillis();
    for (int i = 0; i < 30; i++) { // Go the element 29
        next();
    }
    addIter(20); // Add 20 at the position 29
    fin = System.currentTimeMillis();
    System.out.println("Temps passé: " + (fin - debut) + "ms");
}
```

Conclusions

Nous avons eu l'opportunité de tester onze des treize fonctions, procédures et variables proposées dans l'énoncé. Mais dès le lancement du projet, nous avons été confrontés à plusieurs obstacles.

Pour commencer, parlons du lancement du projet. Parmi les membres du groupe, le poste de travail utilisé n'était pas le même, le système d'exploitation non plus (Fedora Workstation 43 et Windows 11 Home). Donc, par exemple, « *navigator* » n'a pas été reconnu sur l'un des trois postes de travail utilisés.

Par la suite, vers le milieu du temps imparti pour réaliser notre projet, une nouvelle version de la librairie a été éditée. Par manque de temps, nous n'avons pas eu la possibilité d'exécuter de nouveau les tests qui avaient déjà abouti, donc le rapport produit pourrait potentiellement comporter des informations désormais erronées.

Dans l'état actuel de nos connaissances et compétences, nous n'avons pas été en mesure de tenir le délai de trois « heures-personnes ». Néanmoins, nous comprenons mieux l'utilisation de fonctions et procédures, notamment quand on les importe d'une librairie externe.

Ce document et le code ont été rédigés à 100 % sans aide de la moindre IA externe à Visual Studio Code.

Table des figures

fig. 1 : code avec rewind....4
fig. 2 : code avec add....5
fig. 3 : code avec remove....7
fig. 4 : résultats de hasNext....8
fig. 5 : code avec hasNext....9
fig. 6 : code avec size....10
fig. 7 : code avec get....11
fig. 8 : code avec set....12
fig. 9 : code avec hasPrevious et previous....13
fig. 10 : code avec addIter....14